Reproducing The Simple Macroeconomics of AI

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from dataclasses import dataclass
from scipy.optimize import curve_fit

plt.rcParams["figure.figsize"] = (8, 5)
```

1 Parameters

We're working with Hulten's theorem which posits the change in TFP is the profit of the share of the economy subject to increased productivity multiplied by the increase in productivity itself.

Acemoglu adapts this to be:

Change TFP = (jobs exposed to AI x tasks where AI is profitable) x (cost savings x hard-task discount)

```
@dataclass
class Params:
    exposure: float = 0.20 #Eloundou et al. 2023
    profitable_share: float = 0.23 #Svanberg et al. 2024
    cost_savings: float = 0.27 #Noy-Zhang; Brynjolfsson '23
    hard_task_discount: float = 0.53 #Acemoglu 2024
    horizon_years: int = 10

#Params(exposure=0.30, cost_savings=0.40) creates new scenario where
only those change and everything sticks to its default above

P = Params()
P
```

```
Params(exposure=0.2, profitable_share=0.23, cost_savings=0.27,
hard_task_discount=0.53, horizon_years=10)
```

```
def tfp_gain_decade(p: Params) -> float:
    """
    Hulten's theorem (task-based): the economy-wide TFP gain from
automating a set of tasks
    is approximately the cost saving on those tasks, weighted by their
share of value added.

        automated_task_share = exposure * profitable_share
        effective_saving     = cost_savings * hard_task_discount
        TFP gain (decade)    = automated_task_share *
effective_saving

    Target to recover: ~0.5-0.7% over the decade (Acemoglu's
```

```
headline).
    """
    automated_task_share = p.exposure * p.profitable_share
    effective_saving = p.cost_savings * p.hard_task_discount
    return automated_task_share * effective_saving


def annualize(decade_gain: float, years: int = 10) -> float:
    """Convert a cumulative multi-year gain into a geometric annual
rate."""
    return (1.0 + decade_gain) ** (1.0 / years) - 1.0

decade = tfp_gain_decade(P)
annual = annualize(decade, P.horizon_years)

print(f"Baseline TFP gain over {P.horizon_years} years :
{decade*100:6.3f}%")
print(f"Annualized TFP gain                 : {annual*100:6.3f}%")

print("Successful replication")
```

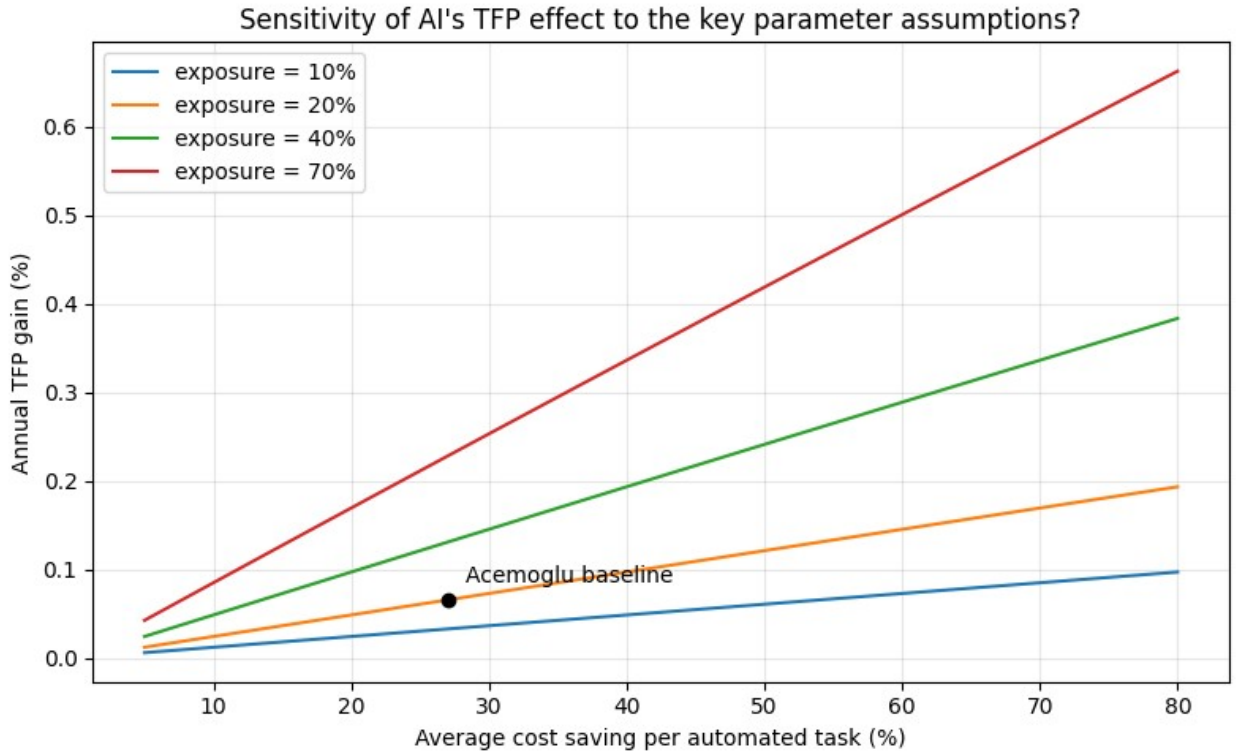
```
Baseline TFP gain over 10 years :  0.658%
Annualized TFP gain                 :  0.066%
Successful replication
```

2 Sensitivity: how the TFP estimate depends on its key assumptions

Acemoglu's headline figure is the product of a few contested parameters. This cell holds the others at baseline and sweeps the two that matter most, the **share of tasks exposed** to AI and the **average cost saving** per automated task. Each line is a fixed exposure level; the x-axis varies cost savings; the y-axis is the resulting annual TFP gain. The black dot marks Acemoglu's own calibration. The takeaway is how *steeply* the conclusion moves with the inputs, and how far you'd have to push them to leave the "modest" regime.

```
# Sweep cost savings across several exposure levels
savings_grid = np.linspace(0.05, 0.80, 100)
exposures = [0.10, 0.20, 0.40, 0.70]

fig, ax = plt.subplots()
for e in exposures:
    gains = [annualize(tfp_gain_decade(
                Params(exposure=e, cost_savings=s,
                       profitable_share=P.profitable_share,
                       hard_task_discount=P.hard_task_discount))) *
100
             for s in savings_grid]
    ax.plot(savings_grid * 100, gains, label=f"exposure = {e:.0%}")

ax.scatter([P.cost_savings * 100], [annual * 100], color="black",
zorder=5)
```

```
ax.annotate("Acemoglu baseline", (P.cost_savings * 100, annual * 100),
            textcoords="offset points", xytext=(8, 8))

ax.set_xlabel("Average cost saving per automated task (%)")
ax.set_ylabel("Annual TFP gain (%)")
ax.set_title("Sensitivity of AI's TFP effect to the key parameter
assumptions?")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



3 Extension; accounting for speed of AI adoption using Anthropic Economic Index Data

5 data release dates, common facet is onet_task_pct this gives us the spread of task distribution, this gives us the shape

Cross ref with Census Business Trends and Outlook Survey to get the level

Then try to replace Acemoglu's borrowed cost saving number with R4 and R5 human_only_time and human_with_ai_time per task

```
import pandas as pd, numpy as np
from huggingface_hub import hf_hub_download   # pip install
huggingface_hub if needed
```

```
REPO = "Anthropic/EconomicIndex"
def hf(path):                                    # downloads (once) +
caches, returns a local path
    return hf_hub_download(REPO, path, repo_type="dataset")

# long-format releases (R3,4,5): facet / variable / cluster_name /
value
def pivot_release(path, facet="onet_task", geography="global"):
    df = pd.read_csv(path)                        # no usecols (that was
the failure point)
    need = {"geography", "facet", "variable", "cluster_name", "value",
"date_start"}
    missing = need - set(df.columns)
    assert not missing, f"missing {missing}; file actually has
{list(df.columns)}"
    sub = df[(df.geography == geography) & (df.facet == facet)]
    sub = sub[~sub.cluster_name.isin(["not_classified", "none"])]
    assert len(sub), f"no rows for facet={facet},
geography={geography}"
    wide = sub.pivot_table(index="cluster_name", columns="variable",
values="value", aggfunc="first")
    return pd.to_datetime(sub["date_start"].iloc[0]), wide

def load_wide(path, date):
    df = pd.read_csv(path)
    df = df[~df.task_name.isin(["not_classified", "none"])]
    wide = df.set_index("task_name")[["pct"]].rename(columns={"pct":
"onet_task_pct"})
    return pd.to_datetime(date), wide

def breadth(wide, top_n=10):
    s = wide["onet_task_pct"].dropna().astype(float); s = s / s.sum()
    return {f"top{top_n}_share":
float(s.sort_values(ascending=False).head(top_n).sum()),
            "hhi": float((s**2).sum()), "n_tasks": int(s.size)}

long_files = [
    hf("release_2025_09_15/data/intermediate/aei_raw_claude_ai_2025-
08-04_to_2025-08-11.csv"),  # R3
    hf("release_2026_01_15/data/intermediate/aei_raw_claude_ai_2025-
11-13_to_2025-11-20.csv"),  # R4
    hf("release_2026_03_24/data/aei_raw_claude_ai_2026-02-05_to_2026-
02-12.csv"),                # R5
]
wide_files = {
    hf("release_2025_02_10/onet_task_mappings.csv"): "2024-12-15",  #
R1
    hf("release_2025_03_27/task_pct_v2.csv"):        "2025-02-15",  #
R2
```

```
}

rows = []
for f in long_files:
    d, w = pivot_release(f); rows.append({"date": d, **breadth(w)})
for f, dt in wide_files.items():
    d, w = load_wide(f, dt); rows.append({"date": d, **breadth(w)})

series = pd.DataFrame(rows).sort_values("date").reset_index(drop=True)
series["A_breadth"] = 1 - series["top10_share"]
print(series.to_string())
```

```
/Users/elyse/Desktop/Acemoglu
AI/.venv/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning:
IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.

        date  top10_share       hhi  n_tasks  A_breadth
0 2024-12-15     0.212503  0.007260     3513   0.787497
1 2025-02-15     0.246131  0.010193     3364   0.753869
2 2025-08-04     0.249703  0.010032     2616   0.750297
3 2025-11-13     0.259288  0.011176     3168   0.740712
4 2026-02-05     0.209107  0.007445     3258   0.790893

/var/folders/rc/8v3nkhlx6snf9cb6lcnglsmc0000gn/T/
ipykernel_15855/3952621550.py:10: DtypeWarning: Columns (0:
cluster_name) have mixed types. Specify dtype option on import or set
low_memory=False.
  df = pd.read_csv(path)                        # no usecols (that was
the failure point)
```

Findings from applying Anthropic Economic Index

Bad news: A_breadth barely fluctuates, A_breadth=1-top_share , if A_breadth=1 the top ten capture basically nothing, if A_breadth=0 the top ten capture everything - it doesn't fluctuate over time meaningfully thus its tells us little about AI adoption

Good news: data pull is consistent with the written report's note that between the last two releases it moved from 0.259 to 0.209 and report records 24% to 19%

Next step: Census Business Trends and Outlook Survey

1st step pandas melt to format from wide to long, its a

4 Anchoring Acemoglu's task-based TFP ceiling to BTOS adoption rates

- Keep the **"Yes"** answers to **Q7** ("used AI in the last two weeks").
- Reshape the period columns wide → long (`melt`) so each row is one survey period.
- Turn the `YYYYPP` period codes into dates and the percentages into fractions.
- Output: `A`, a clean biweekly series of the **share of firms using AI** (~4% → 10%).

```
import pandas as pd

raw = pd.read_excel("AI Core Questions.xlsx")

# 1–2: Q7 + "Yes", then keep just the 6-digit period columns
ai = raw[(raw["Question ID"] == 7) & (raw["Answer"] == "Yes")]
period_cols = [c for c in raw.columns if str(c).isdigit() and
len(str(c)) == 6]
ai = ai[period_cols]

# 3: wide -> long  (THE move)
long = ai.melt(var_name="period", value_name="rate")

# 4: period code YYYYPP -> approx date; clean the rate to a fraction
long["year"] = long["period"].astype(str).str[:4].astype(int)
long["pnum"] = long["period"].astype(str).str[4:].astype(int)
long["date"] = (pd.to_datetime(long["year"].astype(str) + "-01-01")
                + pd.to_timedelta((long["pnum"] - 1) * 14, unit="D"))
# biweekly ≈ 14 days
long["rate"] = pd.to_numeric(long["rate"].astype(str).str.rstrip("%"),
errors="coerce")
if long["rate"].max() > 1:          # if values came in as 6.1 not
0.061
    long["rate"] = long["rate"] / 100

A = long[["date",
"rate"]].dropna().sort_values("date").reset_index(drop=True)
print(A)
```

```
         date   rate
0  2023-09-10  0.037
1  2023-09-24  0.037
2  2023-10-08  0.038
3  2023-10-22  0.039
4  2023-11-05  0.044
5  2023-11-19  0.046
6  2023-12-03  0.049
7  2023-12-17  0.049
8  2024-01-01  0.046
9  2024-01-15  0.053
10 2024-01-29  0.050
11 2024-02-12  0.054
```

```
12 2024-02-26  0.043
13 2024-03-11  0.044
14 2024-03-25  0.042
15 2024-04-08  0.048
16 2024-04-22  0.047
17 2024-05-06  0.048
18 2024-05-20  0.046
19 2024-06-03  0.050
20 2024-06-17  0.047
21 2024-07-01  0.050
22 2024-07-15  0.051
23 2024-07-29  0.051
24 2024-08-12  0.055
25 2024-08-26  0.057
26 2024-09-09  0.059
27 2024-09-23  0.060
28 2024-10-07  0.060
29 2024-10-21  0.060
30 2024-11-04  0.061
31 2024-11-18  0.061
32 2024-12-02  0.061
33 2024-12-16  0.063
34 2025-01-01  0.060
35 2025-01-15  0.067
36 2025-01-29  0.068
37 2025-02-12  0.073
38 2025-02-26  0.074
39 2025-03-12  0.078
40 2025-03-26  0.080
41 2025-04-09  0.083
42 2025-04-23  0.087
43 2025-05-07  0.087
44 2025-05-21  0.092
45 2025-06-04  0.092
46 2025-06-18  0.093
47 2025-07-02  0.093
48 2025-07-16  0.094
49 2025-07-30  0.088
50 2025-08-13  0.097
51 2025-08-27  0.091
52 2025-09-10  0.099
53 2025-09-24  0.100
```

Combine adoption with Acemoglu's ceiling

- `ceiling` = Acemoglu's eventual TFP gain at full adoption (~0.66%).
- `progress` = adoption rate ÷ saturation `L` → how far along we are, in [0,1].
- `tfp_path` = `ceiling × progress` → the TFP gain **realized so far**, period by period.
- `tfp_growth` = its change each period → the per-period productivity contribution.

The ceiling sets the *size* of the prize; adoption sets *how much of it is switched on* yet.

```
import numpy as np

ceiling = tfp_gain_decade(P)                    # scalar, ~0.0066

# A is your cleaned BTOS frame: columns ['date', 'rate'], rate ~0.06 -
> 0.10
# Normalize the firm-adoption RATE into progress in [0,1].
# progress = rate / L, where L = the eventual saturation (share of
firms that ever adopt).
L = 1.0                                          # assumption:
~universal eventual adoption (also try 0.6, 0.8)
progress = (A["rate"] / L).clip(upper=1.0)      # this is A_t, a real
Series now

tfp_path   = ceiling * progress                 # the time series —
runs, because progress is real
tfp_growth = tfp_path.diff()                    # annual-ish growth
contribution

out = A.assign(progress=progress, tfp_path=tfp_path,
tfp_growth=tfp_growth)
print(out[["date", "rate", "progress", "tfp_path"]].tail())
```

```
         date   rate  progress  tfp_path
49 2025-07-30  0.088     0.088  0.000579
50 2025-08-13  0.097     0.097  0.000639
51 2025-08-27  0.091     0.091  0.000599
52 2025-09-10  0.099     0.099  0.000652
53 2025-09-24  0.100     0.100  0.000658
```

As of late September 2025, ~10% of firms use AI, so the model has realized ~0.066% of the TFP gain — roughly one-tenth of AI's eventual ~0.66% effect

Fit the diffusion curve and project forward

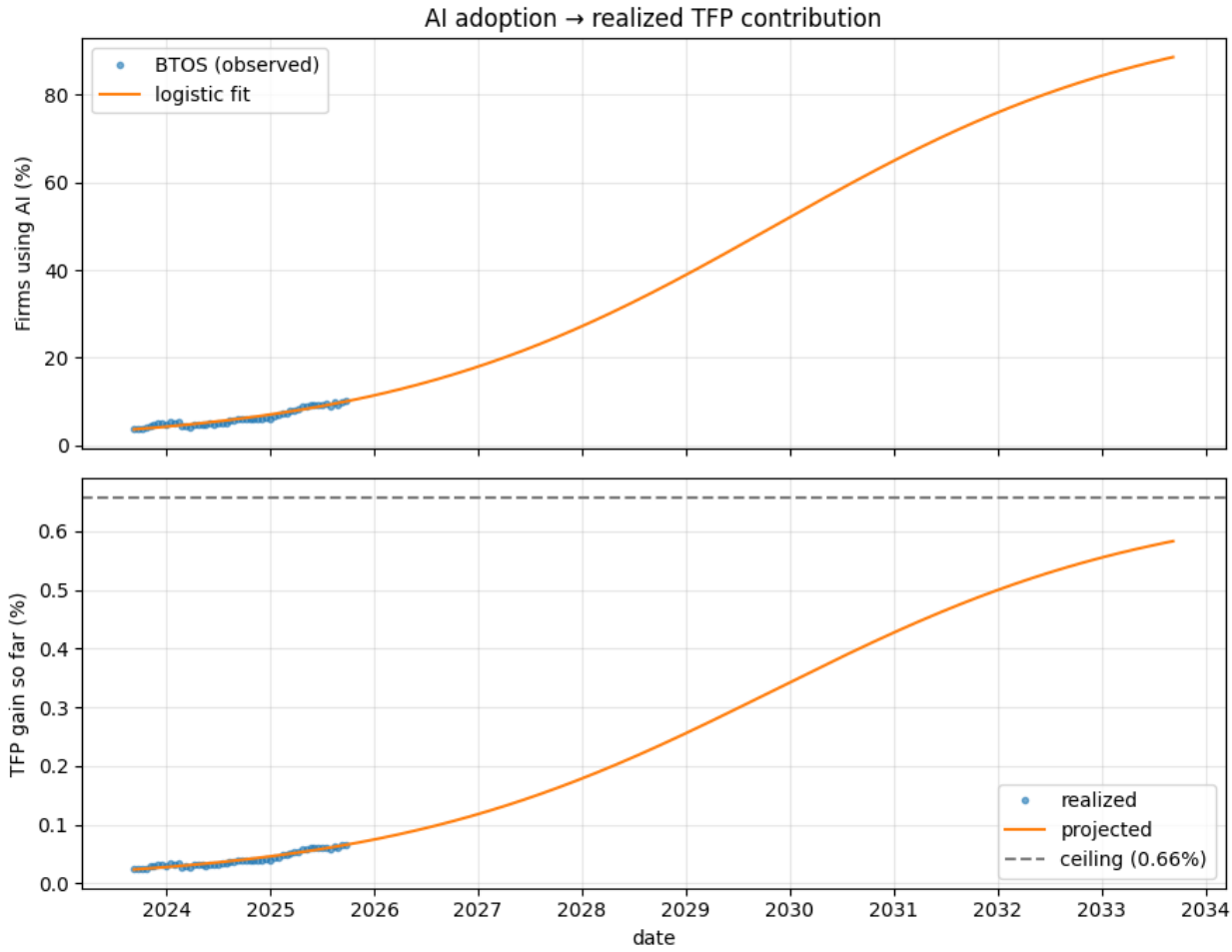
- Fit a **logistic S-curve** to the adoption rate → speed `k` (steepness of the diffusion S-curve (per-year growth rate) found by least-squares fit, well-identified) and saturation `L` (**fragile** — the series is still rising, so the ceiling is a guess).
- Normalize with that same `L`: `progress = rate / L`, `tfp_path = ceiling × progress`.
- Extend the fitted curve 10 years out to project the future adoption rate and TFP path.
- Print the TFP realized now and what share of the ceiling that is.

```
import numpy as np, pandas as pd
from scipy.optimize import curve_fit

t = (A["date"] - A["date"].min()).dt.days / 365.25          # years
since start
```

```
def logistic(t, L, k, t0): return L / (1 + np.exp(-k * (t - t0)))

(L, k, t0), _ = curve_fit(logistic, t, A["rate"].to_numpy(),
                          p0=[0.3, 0.8, 3], bounds=([0.05, 0.01, -5],
[1.0, 10, 20]), maxfev=20000)
print(f"saturation L = {L:.1%}  (FRAGILE)   speed k = {k:.2f}
inflection t0 = {t0:.1f} yrs")

A["progress"] = (A["rate"] / L).clip(upper=1.0)             # SAME L
everywhere
A["tfp_path"] = ceiling * A["progress"]

tf = np.linspace(t.min(), 10, 300)                          # project
10 years out
proj = pd.DataFrame({"date": A["date"].min() +
pd.to_timedelta(tf*365.25, unit="D"),
                     "rate": logistic(tf, L, k, t0)})
proj["progress"] = proj["rate"] / L
proj["tfp_path"] = ceiling * proj["progress"]
print(f"now: {A['tfp_path'].iloc[-1]*100:.3f}% realized —
~{A['progress'].iloc[-1]*100:.0f}% of the {ceiling*100:.2f}% ceiling")
```

```
saturation L = 100.0%  (FRAGILE)   speed k = 0.53   inflection t0 =
6.2 yrs
now: 0.066% realized — ~10% of the 0.66% ceiling
```

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(2, 1, figsize=(9, 7), sharex=True)
ax[0].plot(A["date"], A["rate"]*100, "o", ms=3, alpha=.6, label="BTOS
(observed)")
ax[0].plot(proj["date"], proj["rate"]*100, "-", lw=1.5,
label="logistic fit")
ax[0].set_ylabel("Firms using AI (%)"); ax[0].set_title("AI adoption →
realized TFP contribution"); ax[0].legend(); ax[0].grid(alpha=.3)
ax[1].plot(A["date"], A["tfp_path"]*100, "o", ms=3, alpha=.6,
label="realized")
ax[1].plot(proj["date"], proj["tfp_path"]*100, "-", lw=1.5,
label="projected")
ax[1].axhline(ceiling*100, ls="--", color="gray", label=f"ceiling
({ceiling*100:.2f}%)")
ax[1].set_ylabel("TFP gain so far (%)"); ax[1].set_xlabel("date");
ax[1].legend(); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
```



5 Aggregate to a quarterly productivity shock

- Resample the biweekly `tfp_path` to **end-of-quarter levels** (`QE`, `.last()`).
- Difference consecutive quarters → the **TFP gain added each quarter** (in percentage points).
- This per-quarter increment is the productivity shock the Stage-3 gap model consumes.

```
q = A.set_index("date")["tfp_path"].resample("QE").last()   # end-of-quarter cumulative level
q_growth = q.diff().dropna()                                 # per-quarter increment = the productivity shock
print((q_growth*100).round(4).rename("tfp_growth_pp").tail(8))
```

```
date
2023-12-31    0.0079
2024-03-31   -0.0046
2024-06-30    0.0033
2024-09-30    0.0086
2024-12-31    0.0020
2025-03-31    0.0112
2025-06-30    0.0086
```

```
2025-09-30    0.0046
Freq: QE-DEC, Name: tfp_growth_pp, dtype: float64
```

tfp_path = ceiling × progress q_growth = Δ(tfp_path) = Δ(ceiling × progress) = ceiling × Δprogress

q_growth_quarter = ceiling × Δprogress_quarter = (pi_bar × automatable_share) × (this quarter's change in adoption) = [Acemoglu's ΔTFP formula] × [how much more adoption happened]

6 Saturation scenarios, bracket the unknown ceiling

- `L` (eventual adoption ceiling) is the one parameter the data cannot captur, so we test it explicitly **low / medium / high**.
- For each, refit the curve's speed `k` and recompute how far along we are, the TFP realized so far, and years to near-saturation.
- The table's wide spread is the honest finding: **the same data implies very different conclusions depending on where adoption tops out** ; speed is known, ceiling is a scenario.

```
import numpy as np, pandas as pd
from scipy.optimize import curve_fit

t    = ((A["date"] - A["date"].min()).dt.days / 365.25).to_numpy()
rate = A["rate"].to_numpy()

def fit_fixed_L(L):
    f = lambda tt, k, t0: L / (1 + np.exp(-k * (tt - t0)))
    (k, t0), _ = curve_fit(f, t, rate, p0=[0.5, 3], bounds=([0.01, -
5], [10, 20]), maxfev=20000)
    return k, t0

rows = []
for name, L in {"low": 0.15, "medium": 0.30, "high": 0.60}.items():
    k, t0 = fit_fixed_L(L)
    progress = min(rate[-1] / L, 1.0)
    rows.append({"saturation": f"{name} ({L:.0%})", "speed k":
round(k, 2),
                 "% of the way now": round(progress * 100, 1),
                 "TFP realized now %": round(ceiling * progress * 100,
4),
                 "yrs to 90% adoption": round(t0 + np.log(9) / k, 1)})
print(pd.DataFrame(rows).to_string(index=False))
```

```
  saturation  speed k  % of the way now  TFP realized now %  yrs to
90% adoption
   low (15%)     0.87              66.7              0.4388
4.0
medium (30%)     0.64              33.3              0.2194
6.6
```

```
  high (60%)     0.56                16.7                0.1097
8.8
```

7 Feeding shock into a compact gap model *exclusively illustrative*

- Feeds the per-quarter AI productivity shock into a small 3-equations Phillips, IS, Taylor
- **Mechanism it shows:** AI lowers costs → inflation dips → central bank cuts rates → output booms → reverts as adoption saturates
- ⚠ **Not robust by design:** parameters are assumed, `lam` is cranked for visibility, and it's backward-looking so the *direction* of the channel is the point; the *magnitudes* mean nothing and shouldn't be read as a forecast

```
# projected quarterly AI productivity contribution (medium
saturation), annualized to pp
k, t0 = fit_fixed_L(0.30); L = 0.30
tf = np.linspace(t.min(), 12, 400)
proj_tfp = ceiling * (1 / (1 + np.exp(-k * (tf - t0))))           # =
ceiling * progress
dates_f  = A["date"].min() + pd.to_timedelta(tf * 365.25, unit="D")
g = (pd.Series(proj_tfp,
index=dates_f).resample("QE").last().diff().clip(lower=0).dropna()) *
4 * 100

pi_tar, r_star      = 2.0, 1.0
b_pi, kappa, lam    = 0.6, 0.1, 8.0     # Phillips (lam is
ILLUSTRATIVE!!!)
b_y,  delta, psi    = 0.7, 0.3, 0.0     # IS
rho,  phi_pi, phi_y = 0.8, 1.5, 0.5     # Taylor
n = len(g); yhat = np.zeros(n); pi = np.full(n, pi_tar); i =
np.full(n, r_star + pi_tar)
for s in range(1, n):
    sh = g.iloc[s]
    yhat[s] = b_y*yhat[s-1] - delta*(i[s-1] - pi[s-1] - r_star) +
psi*sh
    pi[s]   = b_pi*pi[s-1] + (1-b_pi)*pi_tar + kappa*yhat[s-1] -
lam*sh
    i[s]    = rho*i[s-1] + (1-rho)*(r_star + pi[s] + phi_pi*(pi[s]-
pi_tar) + phi_y*yhat[s])
resp = pd.DataFrame({"date": g.index, "ai_shock_pp":
g.round(3).values,
                     "inflation": pi.round(2), "output_gap":
yhat.round(2), "policy_rate": i.round(2)})
print(resp.iloc[::4].to_string(index=False))
```

```
      date  ai_shock_pp  inflation  output_gap  policy_rate
2023-12-31        0.051       2.00        0.00         3.00
2024-12-31        0.069       0.85       -0.09         1.68
2025-12-31        0.091       0.34        0.44         0.25
2026-12-31        0.113       0.11        1.18        -0.55
2027-12-31        0.103       0.34        1.68        -0.51
```

2028-12-31	0.069	0.83	1.78	0.14
2029-12-31	0.045	1.32	1.55	1.08
2030-12-31	0.031	1.63	1.12	1.91
2031-12-31	0.017	1.82	0.68	2.46
2032-12-31	0.008	1.91	0.35	2.74
2033-12-31	0.005	1.95	0.17	2.86
2034-12-31	0.003	1.97	0.09	2.92